

```

const express = require('express');
const fs = require('fs');
const path = require('path');
const twilio = require('twilio');

const app = express();
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

const DATA_FILE = path.join(__dirname, 'data.json');

// ---- storage helpers ----
function loadJobs() {
  if (!fs.existsSync(DATA_FILE)) return [];
  try {
    return JSON.parse(fs.readFileSync(DATA_FILE, 'utf8'));
  } catch (e) {
    return [];
  }
}

function saveJobs(jobs) {
  fs.writeFileSync(DATA_FILE, JSON.stringify(jobs, null, 2));
}

// ---- twilio ----
function getTwilioClient() {
  const sid = process.env.TWILIO_ACCOUNT_SID;
  const token = process.env.TWILIO_AUTH_TOKEN;
  if (!sid || !token) return null;
  return twilio(sid, token);
}

function getRecipients() {
  const raw = process.env.RECIPIENTS || '';
  return raw.split(',').map(n => n.trim()).filter(Boolean);
}

function buildMessage(job) {
  const lines = [];
  lines.push('NEW SLIP');
  if (job.tripLabel) lines.push(job.tripLabel);
  lines.push(`Date: ${job.dropDate || ''}`);
  lines.push(`Customer: ${job.customer}`);
  lines.push(`Address: ${job.address}`);
}

```

```

    if (job.type === 'delivery') {
      if (job.material) lines.push(`Material: ${job.material}`);
      if (job.yards) lines.push(`Yards: ${job.yards} YD`);
    } else if (job.size) {
      lines.push(`Size: ${job.size}`);
    }
    if (job.price) lines.push(`Price: $$${job.price}`);
    lines.push(`Instructions: ${job.notes || 'None'}`);
    return lines.join('\n');
  }
}

async function sendGroupText(message) {
  const client = getTwilioClient();
  const from = process.env.TWILIO_FROM_NUMBER;
  const recipients = getRecipients();

  if (!client || !from) {
    return { sent: false, reason: 'Twilio not configured (missing TWILIO_ACCOUNT_'
  }
  if (recipients.length === 0) {
    return { sent: false, reason: 'No RECIPIENTS configured' };
  }

  const results = await Promise.allSettled(
    recipients.map(to => client.messages.create({ body: message, from, to }))
  );

  const failures = results
    .map((r, i) => ({ r, to: recipients[i] }))
    .filter(x => x.r.status === 'rejected')
    .map(x => ({ to: x.to, error: x.r.reason?.message || 'unknown error' }));

  return { sent: failures.length < recipients.length, failures };
}

function getEmailRecipients() {
  const raw = process.env.EMAIL_RECIPIENTS || '';
  return raw.split(',').map(e => e.trim()).filter(Boolean);
}

async function sendGroupEmail(message, job) {
  const apiKey = process.env.RESEND_API_KEY;
  const from = process.env.EMAIL_FROM;
  const recipients = getEmailRecipients();

  if (!apiKey || !from) {
    return { sent: false, reason: 'Resend not configured (missing RESEND_API_KEY/'

```

```

    }
    if (recipients.length === 0) {
      return { sent: false, reason: 'No EMAIL_RECIPIENTS configured' };
    }

    const subject = job.type === 'delivery' ? 'New Delivery' : 'New Rental';

    const results = await Promise.allSettled(
      recipients.map(to =>
        fetch('https://api.resend.com/emails', {
          method: 'POST',
          headers: {
            Authorization: `Bearer ${apiKey}`,
            'Content-Type': 'application/json',
          },
          body: JSON.stringify({
            from,
            to,
            subject,
            text: message,
          }),
        }).then(async (res) => {
          if (!res.ok) {
            const body = await res.text();
            throw new Error(`${res.status}: ${body}`);
          }
          return res.json();
        })
      )
    );

    const failures = results
      .map((r, i) => ({ r, to: recipients[i] }))
      .filter(x => x.r.status === 'rejected')
      .map(x => ({ to: x.to, error: x.r.reason?.message || 'unknown error' }));

    return { sent: failures.length < recipients.length, failures };
  }

  // ---- routes ----
  app.get('/api/jobs', (req, res) => {
    const jobs = loadJobs().sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt));
    res.json(jobs);
  });

  const TRUCK_CAPACITY_YARDS = 5;

```

```

app.post('/api/jobs', async (req, res) => {
  const { type, customer, address, size, material, yards, dropDate, pickupDate, p

  if (!customer || !address) {
    return res.status(400).json({ error: 'Customer and address are required' });
  }

  const isDelivery = type === 'delivery';
  const yardsNum = isDelivery ? parseInt(yards, 10) || 0 : 0;
  const tripsNeeded = isDelivery && yardsNum > TRUCK_CAPACITY_YARDS
    ? Math.ceil(yardsNum / TRUCK_CAPACITY_YARDS)
    : 1;

  const jobs = loadJobs();
  const trips = [];

  for (let i = 0; i < tripsNeeded; i++) {
    let tripYards = null;
    if (isDelivery) {
      const remaining = yardsNum - i * TRUCK_CAPACITY_YARDS;
      tripYards = Math.min(TRUCK_CAPACITY_YARDS, remaining);
    }

    const job = {
      id: Date.now().toString(36) + Math.random().toString(36).slice(2, 6) + '-',
      type: type || 'rental',
      customer,
      address,
      size: isDelivery ? '' : (size || ''),
      material: isDelivery ? (material || '') : '',
      yards: isDelivery ? tripYards : null,
      dropDate: dropDate || '',
      pickupDate: isDelivery ? '' : (pickupDate || ''),
      price: i === 0 ? (price || '') : '',
      notes: notes || '',
      tripLabel: tripsNeeded > 1 ? `Trip ${i + 1} of ${tripsNeeded}` : '',
      createdAt: new Date().toISOString(),
    };

    const message = buildMessage(job);

    let twilioResult;
    try {
      twilioResult = await sendGroupText(message);
    } catch (e) {
      twilioResult = { sent: false, reason: e.message };
    }
  }
}

```

```
    let emailResult;
    try {
      emailResult = await sendGroupEmail(message, job);
    } catch (e) {
      emailResult = { sent: false, reason: e.message };
    }

    const textResult = {
      sent: twilioResult.sent || emailResult.sent,
      twilio: twilioResult,
      email: emailResult,
    };

    jobs.push({ ...job, textResult });
    trips.push({ job, textResult, message });
  }

  saveJobs(jobs);
  res.json({ trips });
});

app.delete('/api/jobs/:id', (req, res) => {
  const jobs = loadJobs().filter(j => j.id !== req.params.id);
  saveJobs(jobs);
  res.json({ ok: true });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Dumpster SMS app running on port ${PORT}`));
```